CODEFORCES
Sponsored by Telegram

2 |
spineight | Выйти

ГЛАВНАЯ ТОП КАТАЛОГ СОРЕВНОВАНИЯ ТРЕНИРОВКИ АРХИВ ГРУППЫ РЕЙТИНГ EDU API КАЛЕНДАРЬ ПОМОЩЬ

IGORJAN94 БЛОГ КОМАНДЫ ПОПЫТКИ ГРУППЫ СОРЕВНОВАНИЯ

Блог пользователя Igorjan94

C++17, competitive programming edition

Автор **Igorjan94**, история, 4 года назад, перевод,

> C++17 уже доступен на codeforces, сообщество хочет новую версию C++ tricks, которую написал **HosseinYousefi**, так что, начнем!
> Disclaimer: Я сделал всего лишь немного примеров новых фич, которые по моему мнению относятся к спортивному программированию. Если у Вас есть примеры лучше или Вам что-то непонятно, или нужно больше объяснений каких-то фич — пишите в комментах)
>
> ## Fold expressions (Свертки)
>
> - Я думаю все знают, что такое reduce и свертка, но все-таки приведу пример из c++11:
>
> ```
> vector<int> v = {1, 3, 5, 7};
> int res = accumulate(v.begin(), v.end(), 0, [](int a, int b) { return a + b; });
> cout << res; // 16
> ```
>
> - Начиная с C++17 есть поддержка свертки для шаблонного списка со следующим синтаксисом:

→ Обратите внимание

До соревнования
Codeforces Round #804 (Div. 2)
40:46:53
Зарегистрироваться »
*есть доп. регистрация

6

→ spineight

Рейтинг: 937
Вклад: 0

- Настройки
- Блог
- Команды
- Попытки
- Избранное
- Группы
- Переписка
- Соревнования

spineight

→ Лидеры (рейтинг)

№	Пользователь	Рейтинг
1	tourist	3671

```
(pack op ...)
(... op pack)
(pack op ... op init)
(init op ... op pack)
```

- Для примера напишем функцию, которая принимает переменное число аргументов и считает их сумму. До C++17 мы не могли этого сделать без явной передачи первого аргумента:

```
//C++14
auto Sum()
{
    return 0;
}

template<typename Arg, typename... Args>
auto Sum(Arg first, Args... rest)
{
    return first + Sum(rest...);
}

cout << Sum(1, 2, 3, 4, 5); // 15

//C++17
template<typename... Args>
auto Func(Args... args)
{
    return (args + ...);
}

cout << Func(1, 2, 3, 4, 5); // 15
```

2	jiangly	3653
3	Um_nik	3629
4	Benq	3513
5	ksun48	3486
6	MiracleFaFa	3466
7	slime	3452
8	maroonrk	3422
9	Radewoosh	3406
10	greenheadstrange	3393

Страны | Города | Организации Всё →

→ Лидеры (вклад)

№	Пользователь	Вклад
1	awoo	187
2	-is-this-fft-	183
3	YouKn0wWho	182
4	Um_nik	179
5	Monogon	177
6	antontrygubO_o	171
7	maroonrk	165
8	SlavicG	163
8	SecondThread	163
8	adamant	163

Всё →

- Это особенно полезно, когда мы в качестве `op` используем запятую:

```
// C++17
template<typename T, typename... Args>
void pushToVector(vector<T>& v, Args&&... args)
{
    (v.push_back(forward<Args>(args)), ...);
    //Этот код раскрывается в последовательность выражений через запятую:
    //  v.push_back(forward<Args_1>(arg1)),
    //  v.push_back(forward<Args_2>(arg2)),
    //  ....
}

vector<int> v;
pushToVector(v, 1, 4, 5, 8);
```

- И мой любимый пример:

```
//C++17
template<typename... Args>
void readln(Args&... args)
{
    ((cin >> args), ...);
}

template<typename... Args>
void writeln(Args... args)
{
    ((cout << args << " "), ...);
}

int x;
```

→ Избранные группы

№	Название
1	Паша и алгосы

Всё →

→ Найти пользователя

Хэндл:

Найти

→ Прямой эфир

faild-in-2021 → java limits

Um_nik → Things I don't know

ChamanDhattarwal → Which algorithm to use to find the shortest path to get a girlfriend?

giga_chad → Hiring bar has reduced for Females at google?

AverageDiv1Enjoyer → rip techno

bijju74 → Need help in debugging this error.

cadmiumky → Codeforces Round #804 (Div. 2)

JaySharma1048576 → Codeforces Round #785 (Div. 2) Editorial

Mr.Nobody → Ok

flamestorm → Codeforces Round #803 (Div. 2)

```
double y;
readln(x, y); // enter 100 500.1234
writeln(x, "some string", y); // 100 some string 500.1234
```

- **Note**: скобки значимы!

Class template argument deduction (Вывод шаблонных типов)

```
template<typename T>
struct point
{
    T x;
    T y;
    point(T x, T y) : x(x), y(y) {}
};

//C++11
pair<int, double> p1 = {14, 17.0}
point<int> u = {1, 2};

//C++17
pair p2 = {14, 17.0}
point v = {1, 2};
```

Если структура сложная, то есть возможность указать правило вывода самим, например так:

```
template<typename T, typename U>
struct S
{
    T first;
```

Editorial

Dinanath_Sv → Tree problem

Wind_Eagle → Разделяй и властвуй. Dynamic connectivity offline и ускорение ДП. Разделяй и властвуй по запросам.

AmmarDab3an → Common reasons for NO verdict in CP assuming your solution is correct on paper

ChamanDhattarwal → Who is the most Mysterious, Inspiring and Captivating person on Codeforces?

adityagamer_alt → HELP: Runtime error on test 2

chokudai → AtCoder Beginner Contest 258 Announcement

Nurss → Tuymaada 2022 Day1

TheOpChicken123 → IOI vs Codeforces: The difficulty and type of questions

peti1234 → Codeforces Round #783 Editorial

Aris → Codeforces Round #797 (Div. 3) Разбор

awoo → Разбор Educational Codeforces Round 129

radoslav11 → CodeChef July Cook-Off (Rated for All) — 3rd July

E869120 → [Tutorial] A Way to Practice Competitive Programming : From rating 1000 to 2000

maroonrk → AtCoder Regular Contest 143 Announcement

baryon → Find any string of length N corresponding to the given set of three-letter substrings

Детальнее →

```
    U second;
};

// Мой вывод типа
template<typename T, typename U>
S(const T &first, const U &second) -> S<T, U>;
```

Note: компилятор обычно сам может создать правило вывода из конструктора, но в этом примере конструктора нет, поэтому правило вывода написано руками.

`*this` capture in lambda expressions (Захват `*this` в лямбда-функциях)

Я не думаю, что это особо полезно в спортивном программировании, но кто знает:

```
struct someClass
{
    int x = 0;

    void f() const
    {
        cout << x << '\n';
    }

    void g()
    {
        x++;
    }

    // C++14
```

↑ 6 ↓

```
    void func()
    {
        auto lambda1 = [self = *this]() { self.f(); };
        auto lambda2 = [self = *this]() mutable { self.g(); };
        lambda1();
        lambda2();
    }

    // C++17
    void funcNew()
    {
        auto lambda1 = [*this]() { f(); };
        auto lambda2 = [*this]() mutable { g(); };
        lambda1();
        lambda2();
    }
};
```

Статья о ключевом слове `mutable` .

Structured bindings (Структурные связывания?)

- Самое полезное нововведение — синтаксический сахар для декомпозиции объектов.

```
template<typename T>
struct point
{
    T x;
    T y;
    point(T x, T y) : x(x), y(y) {}
```

```
};

vector<point<int>> points = {{0, 0}, {1, 0}, {1, 1}, {1, 0}};
//C++11
for (auto& point : points)
{
    int x, y;
    tie(x, y) = point;
    //...Какая-то сложная логика с x и y
}

//C++17
for (auto& [x, y] : points)
{
    //...Какая-то сложная логика с x и y
}
```

- Итерирование по map'у:

```
map<int, string> m;
for (auto [key, value] : m)
    cout << "key: " << key << '\n' << "value: " << value << '\n';
```

- Хорошим примером использования может служить задача 938D - Покупка билетов.
  Код со структурным связыванием (Алгоритм Дейкстры) становится намного понятнее и читаемее: сравните 35474147 и 35346635.

```
while (!q.empty())
{
    auto [dist, u] = *q.begin();
    q.erase(q.begin());
    used[u] = true;
```

```
    for (auto& [w, v] : g[u])
        if (!used[v] && d[v] > dist + 2 * w)
            q.erase({d[v], v}),
            d[v] = dist + 2 * w,
            q.insert({d[v], v});
}
```

Инициализатор в `if` и `switch`

```
set<int> s;

if (auto [iter, ok] = s.insert(42); ok)
{
    //...
}
else
{
    //`ok` и `iter` доступны в этой области видимости
}
//А здесь недоступны
```

Новые атрибуты

- `[[fallthrough]]` атрибут сообщает о том, что break в данном месте пропущен намеренно:

```
int requests, type;
cin >> requests;
for (int q = 0; q < requests; ++q)
    switch (cin >> type; type) //Используем инициализатор в switch
    {
        case 1:
```

```
            int l, r;
            cin >> l >> r;
            //Обработаем запрос первого типа
            break;
        case 2:
            [[fallthrough]];
            //Предупреждение компилятора будет подавлено!
        case 3:
            int value;
            cin >> value;
            //Обработаем запрос второго и третьего типа.
    }
```

- `[[nodiscard]]` атрибут используется, чтобы показать, что возвращаемое значение функции не может быть отброшено. Может использоваться на типах.

std::optional

```
optional<int> findPath(graph g, int from, int to)
{
    //Find path from `from` to `to`
    if (d[to] != INF)
        return d[to];
    return {}
}

//Проверим, что путь существует
if (auto dist = findPath(...); dist.hasValue())
    cout << dist.value(); //Получим его
else
    cout << -1;
```

```
//Или сразу используем defaultValue, если значение не было установлено
cout << findPath(...).value_or(-1); //Выводит расстояние если оно найдено
и -1 иначе
```

Неконстантное(ый?, ая?) string::data

Для любителей C:

```
string str = "hello";
char *p = str.data();
p[0] = 'H';
cout << str; // Hello
```

Свободные функции std::size, std::data и std::empty

В добавку к уже существующим свободным функциям std::begin, std::end и другим, появились новые, такие как: std::size, std::data и std::empty:

```
vector<int> v = { 3, 2, 5, 1, 7, 6 };

size_t sz = size(v);
bool empty = empty(v);
auto ptr = data(v);
```

std::clamp

Возвращает `x` , если оно попало в интервал `[low, high]` и ближайшее значение иначе:

```
cout << clamp(7, 0,  10); //7
cout << clamp(7, 0,  5);  //5
```

```
cout << clamp(7, 10, 50); //10
```

Я думаю, что это полезная функция, но будет сложно вспомнить как она называется в течение контеста :)

GCD and LCM! (НОД и НОК)

```
cout << gcd(24, 60); // 12
cout << lcm(8, 10);  // 40
```

Возвращаемое значение у `emplace_back`

```
vector<int> v = { 1, 2, 3 };

auto &r = v.emplace_back(10);
r = 42;
//v теперь содержит {1, 2, 3, 42}
```

Функции в std::map:

- Extract (можно даже поменять ключ!!!)

```
map<int, string> myMap{ { 1, "Gennady" }, { 2, "Petr" }, { 3, "Makoto" }
};
auto node = myMap.extract(2);
node.key() = 42;
myMap.insert(move(node));

// myMap: {{1, "Gennady"}, {42, "Petr"}, {3, "Makoto"}};
```

Note: Extract — единственный способ поменять ключ элемента map'а без reallocation(реаллокации?)

Асимптотика:
extract(key): $O(\log(N))$ doc
extract(iterator): $O(1)$ amortized doc

- Merge

```
map<int, string> m1{ { 1, "aa" }, { 2, "bb" }, { 3, "cc" } };
map<int, string> m2{ { 4, "dd" }, { 5, "ee" }, { 6, "ff" } };
m1.merge(m2);
// m1: { {1, "aa"}, {2, "bb"}, {3, "cc"}, {4, "dd"}, {5, "ee"}, {6, "ff"}
}
// m2: {}
```

Асимптотика: $O(N\log(N+M))$ doc

- Раньше, чтобы понять, произошла вставка в map или обновление, необходимо было сначала найти элемент, а потом использовать operator[]. Теперь появилась функция insert_or_assign:

```
map<int, string> m;
m.emplace(1, "aaa");
m.emplace(2, "bbb");
m.emplace(3, "ccc");

auto [it1, inserted1] = m.insert_or_assign(3, "ddd");
cout << inserted1; // 0

auto [it2, inserted2] = m.insert_or_assign(4, "eee");
cout << inserted2; // 1
```

Асимптотика: $O(\log(N))$ doc

Более строгий порядок вычисления выражений

В C++17 появились новые правила, более строго определяющие порядок вычисления выражений:

- Постфиксные выражения вычисляются слева направо (в том числе вызовы функций и доступ к членам объектов)
- Выражения присваивания вычисляются справа налево.
- Операнды операторов << и >> вычисляются слева направо.

Таким образом, как указывается в предложении к стандарту, в следующих выражениях теперь гарантированно сначала вычисляется a, затем b, затем c:

```
a.b
a->b
a->*b
a(b1, b2, b3)
b @= a
a[b]
a << b << c
a >> b >> c
```

Note: Порядок вычисления b1, b2, b3 все еще не определен.

P.S.: Все материалы адаптированы мной с примерами отсюда

c++, c++14, c++17

+415 Igorjan94 4 года назад 39

Комментарии (39)

Написать комментарий?

farmersrice

4 года назад, # | +59

Thank goodness for clamp. No more max of min of max of...

→ Ответить

4 года назад, # | 0

Structured bindings are truly awesome!

quachtridat

Thanks for this great article anyway.

Also here is the cppreference link to the page that presents compiler support for new C++ features. These include C++11, C++14, C++17, and later accepted revisions to the standard, as well as various technical specifications.

→ Ответить

4 года назад, # | ← Rev. 3 0

haleyk100198

```
//C++17
template<typename... Args>
void readln(Args&... args)
{
    ((cin >> args), ...);
}

template<typename... Args>
void writeln(Args... args)
{
    ((cout << args << " "), ...);
}

int x;
```

```
int y;
```

Alternatively if you don't want to overkill too hard:

copy(istream_iterator<T>(cin), istream_iterator<T>(), back_inserter(input));

copy(output.begin(), output.end(), ostream_iterator<T>(cout, " "));

Were there some issues hindering __gcd? Or is it just compiler dependent so they introduced gcd as the standard instead?

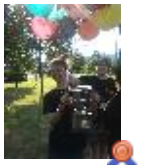
→ Ответить

quachtridat

4 года назад, # ^ | 0

I think this comment may be the answer for your question in the last sentence.

→ Ответить



Igorjan94

4 года назад, # ^ | 0

No, unfortunately ostream_iterator must be instantiated with type:

```
vector<int> myvector;
copy(myvector.begin(), myvector.end(),
ostream_iterator<int>(cout, " "));
```

But my implementation works with different types without any type-mentioning:

```
int x;
double d;
vector<int> v(10); //and overloaded ostream&
```

```
operator<<(ostream& os, vector<T> a) and operator>>
readln(x, d, v);
writeln(x, d, v);
```

→ Ответить

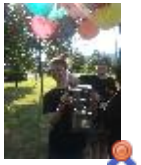
haleyk100198

4 года назад, # ^ | ← Rev. 2 0

ah, yes.

Edit: <T> got considered as a tag :(

→ Ответить



Igorjan94

4 года назад, # ^ | 0

Yes, do not forget, that code `must` be in ```codeBlock``` or `codeLine`, otherwise everything in chevrons disapears :)

→ Ответить

NursultanTolshybek

4 года назад, # | ← Rev. 2 -34

→ Ответить

Комментарий скрыт по причине большого числа негативных отзывов о нем, нажмите здесь для его просмотра

HosseinYousefi

4 года назад, # | +37

“community wants new edition of C++ tricks by Swift,...”

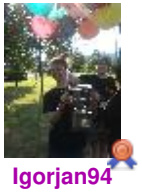
→ Ответить

I_Love_Tina

4 года назад, # | ☆ ← Rev. 2 0

What is the complexity of merging two maps?

→ Ответить



Igorjan94

4 года назад, # ^ | ☆ +1

$O(N \log(N+M))$ doc

→ Ответить

Adhami

4 года назад, # | -20

Can you add the complexity of the new functions?

→ Ответить

Igorjan94

4 года назад, # ^ | 0

Added complexity of new map functions

→ Ответить

Adhami

4 года назад, # ^ | -22

What about others :size(),empty(),data(),gcd(),lcm(),clamp()

I know than I can find the answer by googling but it wouldn't be better if you added them

→ Ответить

Igorjan94

4 года назад, # ^ | 0

`:size(), empty(), data()` are only wrappers, so they have no complexity: complexity is in usual functions `.size()` and so on

`clamp` is literally two comparisons

`gcd` is literally `__gcd` and usual euclidean algo

`lcm` is literally one multiplication and division

So, nothing strange and breaking in these functions, only shortness and convenience

→ Ответить

↑ 6 ↓

4 года назад, # ^ | -22

Bugman

новый, 14 месяцев назад, # ^ 0|

for minusers: check cute `std::is_permutation`

→ Ответить

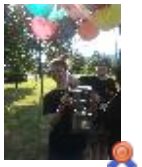
4 года назад, # | 0

zamazan4ik

Если уж мы про олимпиадный подход, то как же ты забыл о новых поисковых алгоритмах? :-) http://en.cppreference.com/w/cpp/algorithm/search

Теперь быстрый поиск уже встроен в кресты: Бойер-Мур и Бойер-Мур-Хорспулл. Реализации ожидаемо должны быть похожи на те, что есть в Boost.Algorithm.

→ Ответить



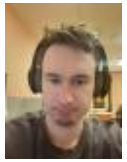
Igorjan94

4 года назад, # ^ | 0

Я не забыл, я специально пропустил, потому что "it has O(nm) in the worst case", а в спортивном программировании всегда worst case :)

→ Ответить

4 года назад, # ^ | ← Rev. 2 -13



adamant

4 года назад, # ^ | 0

Да ладно, если кому-то совсем лень самому написать префикс- или z- функцию, что займёт жалких три строчки, можно использовать обычный string::find. Его сложность хоть и unspecified, но я пока задач, где он валится не встречал.

Бойер-Мур это вообще ни разу не олимпиадный подход.

→ Ответить

knayan

4 года назад, # | 0

```
Why long is of 4 bytes in Codeforces(C++17) while it is of 8
bytes in Codechef(C++14) ?
                Codeforces          Codechef
             (C++17)(4 bytes)    (C++14)(8 bytes)
LONG_MIN :    -2147483648        -9223372036854775808
LONG_MAX :     2147483647         9223372036854775807
ULONG_MAX:     4294967295        18446744073709551615
```

→ Ответить

4 года назад, # ^ | 0

CodeForces sandbox is 32-bit so long is 32-bit too

farmersrice

4 года назад, # ^ | 0

long long is always guaranteed 64 bit. long is not

→ Ответить

Onlynagesha

4 года назад, # | 0

A typical benefit of deduction guide is that we can directly write `tuple(x, y, z)` instead of `make_tuple(x, y, z)`, and `pair(x, y)` instead of `make_pair(x, y)`. Five less characters and the new version looks more compact ;)

→ Ответить

Deba

4 года назад, # ^ | ← Rev. 2 +16

But we can just write `{x,y,z}` instead of `make_tuple(x, y, z)` and `{x,y}` instead of `make_pair(x, y)` and that's 10 and 9 characters less respectively.

→ Ответить

riadwaw

4 года назад, # ^ | +15

not in every context

→ Ответить

4 года назад, # ^ | 0

4 года назад, # ^ | +5

Golovanov399

Case 1:

```
auto p = {1, 2};
cout << p.first << " " <<
p.second << "\n"; // c++ treats p
as an initializer_list<int> and
fails in finding its "first"
field
```

Case 2:

```
auto p = pair{1, 2};
cout << p.first << " " <<
p.second << "\n"; // stdout: 1 2
```

Case 3:

```
auto p = pair(1, 2);
cout << p.first << " " <<
p.second << "\n"; // stdout: 1 2
```

→ Ответить

4 года назад, # ← ^ | Rev. 4 0

4 года назад, # ^ | ← Rev. 2 0

yassin_

Comparisons

```
if (p == {x, y}) ...
```

doesn't work

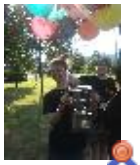
→ Ответить

Igorjan94

4 года назад, # | 0

*Автокомментарий: текст был обновлен пользователем **Igorjan94** (предыдущая версия, новая версия, сравнить).*

→ Ответить



Igorjan94

4 года назад, # | 0

*Auto comment: topic has been updated by **Igorjan94** (previous revision, new revision, compare).*

→ Ответить

downvoteplz

4 года назад, # | ← Rev. 2 0

Here are two more interesting things that should be noted for completeness:

1. `gcd` does not show the same behaviour as `__gcd`, gcd of negative numbers is returned as the positive counterpart. Similarly for `lcm`, LCM of negative numbers is a positive value. So `gcd(-2, -10) = 2, lcm(-2, -10) = 10`. Verifiable here: 36998215
2. `random_shuffle` no longer supports custom generators(for example, using `mt19937` etc.), `shuffle`, however is supported. More about this.

→ Ответить

Hd7

новый, 19 месяцев назад, # | -12

Is space required in structure binding? I noted the different behavior on AtCoder between `for(auto [x, y]:vii)` and `for(auto [x,y]: vii)`. It's so weird. See 2 submissions: 18566943 vs. 18566919.

→ Ответить

ivan100sic

новый, 19 месяцев назад, # ^ | 0

You have undefined behavior on line 64 in `!visc[a[x][y]-'a']` when `a[x][y]` is the letter S, it has nothing to do with the space.

→ Ответить

новый, 19 месяцев назад, # ^ | -13

dohoon.

новый, 19 месяцев назад, # | ← Rev. 2 0

I was wondering which part of C++17 would be appropriate to use for competitive programming, which gave me a good answer :)

→ Ответить

Thallium54

новый, 18 месяцев назад, # | 0

Also I think `try_emplace()` method in `map` is also useful. Using this, coord-compression could be written as

```
for (auto i : a) {
    compress.try_emplace(i, compress.size());
}
```

→ Ответить

↑ 6 ↓

Codeforces (c) Copyright 2010-2022 Михаил Мирзаянов
Соревнования по программированию 2.0
Время на сервере: 03.07.2022 00:48:03 (k1).
Десктопная версия, переключиться на мобильную.
Privacy Policy

При поддержке

УНИВЕРСИТЕТ ИТМО